



# MUSUBI

Paired-LLM software engineering — vibe coding, with receipts.

Two AI builders from two different vendors — one Claude, one Codex — working as colleagues on one shared comms thread, with a single human directing. *Musubi* (結び) is the Shinto idea of binding distinct things into a whole. An honest experiment: built on two real production codebases, driven by one operator.

# TWO SHAPES – THE DIFFERENCE IS WHO PLAYS THE SENIOR ENGINEER

## THE PAIR (OPUS + CODA)

Two builders who write code and review each other. *You* are the senior engineer in the room: you hold the vision, catch the discipline they skip, and judge when they disagree. Maximum control – the engineering conscience is yours to carry.

## THE PAIR + OYA

Add Oyakata (親方, "master craftsman") as the third seat. She sits above the bench the way a master watches two apprentices: she writes no code, she holds the whole picture, and she speaks only when speaking matters.

# OYA – GUARDIAN OF INTENTION

Her first duty is custody of the vision. She holds two pictures at once: the **end state** – the vision, architecture, and strategy you set out to build – and the **two agents** at the bench right now.

## THE PROBLEM SHE SOLVES

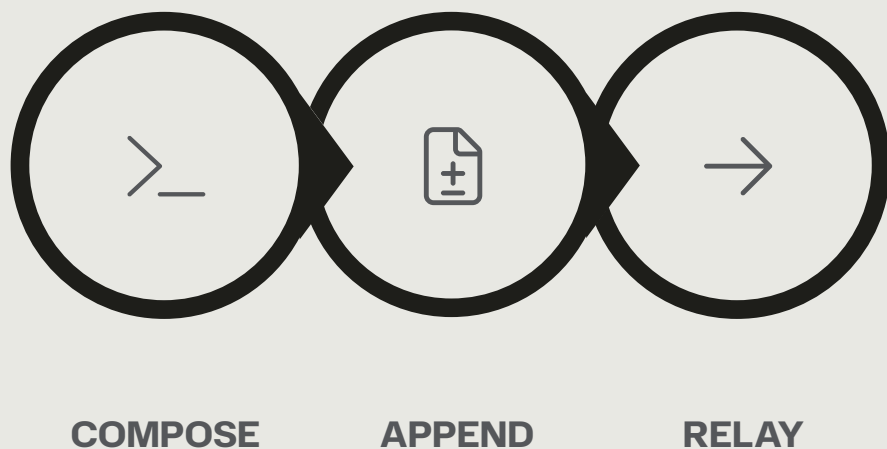
The pair, heads-down in the diff, will cheerfully build a flawless version of the *wrong* thing. Oya is the one still holding what "right" was meant to be – and she steers them back before it ships, not after.

## HOW SHE STARTS

She boots on the vision, architecture, and roadmap docs before she watches anything. If those don't exist, her first act is to say so. She refuses to supervise blind.



# THE BENCH – HOW THE WORKSHOP RUNS



The file is the authority; the relay is just the doorbell.

Opus and Coda work in adjacent terminal panes. A relay watches one shared, append-only comms file. When an agent ends a message with `<OVER>`, the relay carries it to the other – exactly as if a human had typed it.

- The human stays in the loop throughout: watching, intervening, and approving every merge.

# WHY TWO UNLIKE MAKERS

Two builders cut from the same model share the same blind spots — same training, same priors, same misses. Two from *different* vendors do not. Their disagreements surface bugs that two instances of one model would both wave through.

An eight-week audit of paired-cycle output found a category of catches that *only* the cross-vendor reviewer produced — consistently, across cycles. Honest scope: one corpus, one operator. A finding worth defending, not yet a law.



# WHERE MUSUBI SITS

| Pattern                    | Example             | Inherited blind spot                                |
|----------------------------|---------------------|-----------------------------------------------------|
| Solo agent                 | Cursor, Claude Code | Operator is everything; no outside view             |
| Same-vendor sub-agents     | Agent Teams         | Shared context, no external perspective             |
| Same-vendor multi-agent    | LangGraph, AutoGen  | Same training distribution → same blind spots       |
| Multi-vendor pair          | loop CLI            | Closes the vendor gap; no overseer above the pair   |
| <b>musubi — the pair</b>   | this framework      | Multi-vendor pair + operator-led oversight          |
| <b>musubi — pair + Oya</b> | this framework      | Multi-vendor pair + a cross-vendor master craftsman |

The orchestration surface — panes, relay, comms file — is commoditised. The real bet sits below it.

# WHAT THE MASTER CRAFTSMAN WATCHES FOR

**1**

## **STILL SERVING THE GOAL?**

Past "does the code work" – does this slice move toward the end state, or has the work quietly drifted?

**2**

## **CUTTING CORNERS?**

A threat model on an auth change, a rollback on a migration, the error states on a new screen – the discipline a senior would insist on.

**3**

## **WHO WAS RIGHT, AND WHY?**

She watches which of the two unlike builders was right – the whole reason you run two.

**4**

## **ARE YOU DRIFTING TOO?**

If you wave something through or overrule the pair, she notes it – a sanity-check on the human, not just the agents.



# SHE WARNS; SHE DOESN'T TAKE THE WHEEL

## WHY SHE NEVER BLOCKS

A craftsman who *blocks* every cut gets ignored – by the twentieth cycle every block is waived without a thought, and the signal is gone. Oya warns, notes, and logs; she never freezes the work. An honest skip is *data*; a theatre block is *noise*.

## OVERVIEW, NOT CONDUCTOR

She watches and speaks – she does not direct. The pair still self-coordinate, and you are still the gate. A third agent that *directs* the other two is merely the supervisor pattern every framework already ships.

# THE PROTOCOL AUDITS ITS OWN RULES

Discipline lives in a named rule ledger – rules with Mechanism: blocks in the runbook, not buried in a prompt. Every cycle, each rule's fires, catches, skips, and silent misses are logged, so patterns a single review could never see surface over time.

- ❏ Six missed observability checks in a row across one auth slice – visible only in the ledger.

## THE HONEST CLAIM IS NARROW

Two real codebases, one operator.

## THE PAIR IS BATTLE-TESTED

The core has earned its place across real production cycles.

## OYA EARNS HER PLACE

The newer layer – she earns her seat by what she catches.

## VIBE CODING, WITH RECEIPTS

The ledger is the proof.